

On SAT-Encodings of the At-Most-One Constraint

Steffen Hölldobler and Van-Hau Nguyen

International Center for Computational Logic
Technische Universität Dresden, 01062 Dresden, Germany
{sh,hau}@iccl.tu-dresden.de

Abstract. One of the most widely used constraint during the process of translating a practical problem into a propositional satisfiability instance is the at-most-one constraint. This paper proposes a new encoding for the at-most-one constraint, the so-called *bimander* encoding which can be easily extended to encode cardinality constraints. Experimental results reveal that the new encoding is competitive. We prove that the *bimander* encoding allows unit propagation to achieve arc consistency. Furthermore, we show that a special case of the *bimander* encoding outperforms the widely used *binary* encoding in all our experiments.

1 Introduction

Solving propositional satisfiability (SAT) problems is one of the most successful automated reasoning methods in the last decade in Computer Science by solving a wide range of both industrial and academic problems [27,13]. SAT solving comprises two essential phases: encoding a certain problem into a SAT instance and, thereafter, finding solutions by advanced SAT solvers. Notwithstanding the steadily increasing diffusion and availability of SAT solvers, understanding of SAT encodings is still limited and challenging [25].

An increasing number of real-world applications in computer science can be expressed as constraint satisfaction problems (CSPs) [36]. While CSPs can be solved directly using appropriate solvers, the generality and success of SAT solvers in recent years has led to a fruitful competition between the CSP and the SAT community. To utilize state-of-the-art SAT solvers, CSPs need to be encoded as SAT instances (see e.g. [41,5,32,40,39,35,3]). Such encodings should not only be efficiently generated, but should also be efficiently solved by SAT solvers. Currently, mapping a CSP into a SAT instance is regarded more of an art than a science [41,21,31,30,25,35], and detailed studies of different encodings are needed in order to better understand these mappings.

Generally, different encodings of CSPs into SAT instances yield different sizes and different run time behaviour of the used SAT solver. There does not seem to be general knowledge why a particular encoding performs better than others. In this study, we will compare different encodings with respect to the following features:

- the number of auxiliary variables required,
- the number of clauses required,
- the number of variables per clause,
- the strength of the encoding in terms of performance of unit propagation,
- the runtime of a SAT solver on benchmark problems.

Although many encodings have been proposed [41,21,5,32,40,39,35], the most straightforward way of mapping a CSP into a SAT instance is the *direct encoding* (see [41]). The *direct encoding* requires the translation of global constraints like the at-least-one (ALO) and at-most-one (AMO) constraints requiring that a CSP variable has at least one and at most one value assigned to it, respectively. Whereas the ALO constraint can be easily encoded by a single clause, the encoding of the AMO constraint is more complicated and has been intensively studied [26,17,14,34,33]. This is due to the fact that many different applications such as the discrete tomography problem [9], partial Max-SAT [6,7], or cardinality constraints [17] contain the AMO constraint. Furthermore, better encodings of the AMO constraint in conjunctive normal form (CNF) could also help Max-SAT solving, because some state-of-the-art SAT-based Max-SAT solvers encode AMO constraints in CNF [4].

In the direct encoding, if a propositional variable is used to represent the binding of a CSP variable to a particular value, then the AMO constraint requires that at most one of n propositional variables is bound to *TRUE*. Herein, this will be denoted by $\leq_1(X_1, \dots, X_n)$, where X_i , $1 \leq i \leq n$, are propositional variables.

Inspired by many interesting and recent results [26,17,14], especially when Prestwich used the *binary* encoding [18,19] to successfully solve many large instances with a standard SAT solver [34,33], we will survey several widely used encodings of the AMO constraint. In particular, we will point out the similarity of the *relaxed ladder* [33] encoding, the *sequential* [38] encoding, and several others. Finally, we will introduce a new encoding, the *bimander* encoding. The new encoding requires $\lceil \log_2 m \rceil^1$ auxiliary variables and $\frac{n^2}{2m} + n\lceil \log_2 m \rceil - \frac{n}{2}$ binary clauses, where m is the number of disjoint subsets used by dividing the given set $\{X_1, \dots, X_n\}$ of propositional variables.

Additionally, the *bimander* encoding can be easily extended to cardinality constraints, denoted by $\leq_k(X_1, \dots, X_n)$, which expresses that less than k of n propositional variables X_i , $1 \leq i \leq n$, can be simultaneously assigned to *TRUE*. To the best of our knowledge, our encoding is the one that requires the least number of auxiliary variables among known encodings except for the *pairwise* encoding, which needs no auxiliary variables at all. With respect to scalability, the *bimander* encoding can be adjusted by changing the parameter m . For example, by setting the parameter m to specific values, the *binary* and *pairwise* encodings can be expressed as special cases of the *bimander* encoding. Interestingly, the special case of the *bimander* encoding where $m = \lceil \frac{n}{2} \rceil$, outperforms the *binary* encoding in all our experiments. It is important to note that our encoding allows unit propagation (UP) to preserve arc consistency, one of the most important technique in Constraint Programming (see [12]).

The structure of the paper is as follows. In Section 2, we briefly represent many known encodings of the AMO constraint. In Section 3, we describe the new *bimander* encoding and prove several important properties. In Section 4, we compare the *bimander* encoding with other encodings through experiments. Finally, we conclude and outline future research in Section 5.

¹ $\lceil x \rceil$ is the smallest integer not less than x .

2 Existing Encodings

Before giving a brief survey of some of the most widely used AMO encodings, we first define notions and notations, mainly following Frisch and Giannoros [17].

Let $X = \{X_i \mid 1 \leq i \leq n, n \in \mathbb{N}\}$ be a finite set of propositional variables, let A be a finite, possibly empty set of auxiliary propositional variables, and let $\phi(X, A)$ be a propositional formula in conjunctive normal form (CNF) encoding the constraint $\leq_1(X_1, \dots, X_n)$. The encoding $\phi(X, A)$ is *correct* if and only if

- any (partial) assignment $\hat{x}$ that satisfies $\leq_1(X_1, \dots, X_n)$ can be extended to a complete assignment that satisfies $\phi(X, A)$, and
- for any (partial) assignment $\hat{x}$ for X which assigns more than one variable of X to *TRUE*, unit propagation (UP) detects a conflict, i.e., repeated applications of UP yield the empty clause.

UP plays a crucial role in SAT solving as modern SAT solvers spent more than 80% of their runtime on it [15,29], whereas arc consistency is one of the most important techniques in CSP solvers because it is a very good trade-off between the amount and the cost of pruning. Therefore, when translating a CSP to a SAT instance one should pay much attention to determine whether UP on the resulting SAT instance enforces arc consistency. UP of a SAT encoding of the constraint $\leq_1(X_1, \dots, X_n)$ achieves the same pruning as arc consistency on the original CSP if the following holds [17]:

- at most one propositional variable in X is assigned to *TRUE*, and
- if any variable $X_i \in X$ is assigned to *TRUE*, then all the other variables occurring in X must be assigned to *FALSE* using UP.

In the following sections, generally $AMO(X)$, $ALO(X)$, and $EO(X)$ denote the at-most-one, at-least-one, and exactly-one clauses, respectively, for the set of propositional variables X . For the sake of convenience, we will illustrate those encoding on a running example through the set consisting of 8 Boolean variables, $X = \{X_1, \dots, X_8\}$.

2.1 The Pairwise Encoding

This encoding has several different names: the *naive* encoding [38,26], the *pairwise* encoding [37,33], or the *binomial* encoding [17]. In this paper, we refer to it as the *pairwise* encoding. The idea of this encoding is to express that no pair of two variables are simultaneously assigned to *TRUE*, therefore as soon as one literal is assigned to *TRUE*, then all others must be assigned to *FALSE*:

$$\bigwedge_{i=1}^{n-1} \bigwedge_{j=i+1}^n (\overline{X}_i \vee \overline{X}_j)$$

In the running example, the *pairwise* encoding produces the following clauses:

$$\begin{aligned} \overline{X}_1 \vee \overline{X}_2, \overline{X}_1 \vee \overline{X}_3, \overline{X}_1 \vee \overline{X}_4, \dots, \overline{X}_1 \vee \overline{X}_8, \\ \overline{X}_2 \vee \overline{X}_3, \overline{X}_2 \vee \overline{X}_4, \dots, \overline{X}_2 \vee \overline{X}_8, \\ \overline{X}_3 \vee \overline{X}_4, \dots, \overline{X}_3 \vee \overline{X}_8, \\ \vdots \\ \overline{X}_7 \vee \overline{X}_8. \end{aligned}$$

The *pairwise* encoding is a traditional way of encoding the AMO constraint into SAT. Although this encoding does not need any auxiliary variables, it requires a quadratic number of clauses (see Table 1). As a result, this encoding produces impractically large formulas on problems with large domains. Nevertheless, the *pairwise* encoding is not only widely used in practice, but also easily combined with other encodings [26,40,14]. The encoding allows UP to maintain arc consistency.

2.2 The Binary Encoding

Frisch et al. [18,19] proposed the *binary* encoding. Independently, Prestwich called it *bitwise* encoding [33,35]) and used it to successfully solve a number of large instances of CSPs with a standard SAT solver [34,33].

The binary encoding requires new Boolean variables $B_1, \dots, B_{\lceil \log_2 n \rceil}$ with the understanding that B_j (or $\overline{B}_j$) is the bit j of $i - 1$ represented by a binary string is. The desired clauses are

$$\bigwedge_{i=1}^n \bigwedge_{j=1}^{\lceil \log_2 n \rceil} (\overline{X}_i \vee \phi(i, j)),$$

where $\phi(i, j)$ denotes B_j (or $\overline{B}_j$) if bit j of $i - 1$ represented by a binary string is 1 (or 0). The running example is represented by the *binary* encoding as follows:

$$\begin{aligned} \overline{X}_1 \vee \overline{B}_1, \overline{X}_2 \vee B_1, \overline{X}_3 \vee \overline{B}_1, \dots, \overline{X}_8 \vee B_1, \\ \overline{X}_1 \vee \overline{B}_2, \overline{X}_2 \vee \overline{B}_2, \overline{X}_3 \vee B_2, \dots, \overline{X}_8 \vee B_2, \\ \overline{X}_1 \vee \overline{B}_3, \overline{X}_2 \vee \overline{B}_3, \overline{X}_3 \vee \overline{B}_3, \dots, \overline{X}_8 \vee B_3. \end{aligned}$$

The idea is to create the different sequences of $\lceil \log_2 n \rceil$ -tuples $B_j, 1 \leq j \leq \lceil \log_2 n \rceil$, such that whenever any X_i is assigned to *TRUE*, $1 \leq i \leq n$, then we immediately infer that the other variables $X_{i'}$ must be assigned to *FALSE*, for any $1 \leq i' \neq i \leq n$. One should observe that with the binary encoding UP maintains arc consistency.

2.3 The Commander Encoding

Klieber and Kwon [26] described the *commander* encoding by dividing the set $X = \{X_1, \dots, X_n\}$ of propositional variables into m , $1 \leq m \leq n$, disjoint subsets denoted by $\{G_1, \dots, G_m\}$, and introducing a *commander* variable c_i for each subset G_i , $1 \leq i \leq m$. The *commander* encoding is defined as follows:

1. Exactly one variable in each set $G_i \cup \{\bar{c}_i\}$ is assigned to *TRUE*.

$$\bigwedge_{i=1}^m EO(\{\bar{c}_i\} \cup G_i) = \bigwedge_{i=1}^m AMO(\{\bar{c}_i\} \cup G_i) \wedge \bigwedge_{i=1}^m ALO(\{\bar{c}_i\} \cup G_i).$$

Whereas the ALO constraint is easily translated into a single clause, AMO can be encoded either by the pairwise on any other encoding. In the running example, we select $m = 4$ and divide the set $X = \{X_1, \dots, X_8\}$ into the disjoint subsets $G_1 = \{X_1, X_2\}$, $G_2 = \{X_3, X_4\}$, $G_3 = \{X_5, X_6\}$, and $G_4 = \{X_7, X_8\}$. Then, four commander variables c_1, c_2, c_3 , and c_4 are added. We obtain:

$$AMO(\bar{c}_1, X_1, X_2) \wedge (\bar{c}_1 \vee X_1 \vee X_2) \wedge \dots \wedge AMO(\bar{c}_4, X_7, X_8) \wedge (\bar{c}_4 \vee X_7 \vee X_8).$$

2. At most one commander variable is assigned to *TRUE*. This constraint can be encoded either by the *pairwise* encoding or by another encoding (even by a recursive application of the commander encoding):

$$\bigwedge_{i=1}^m AMO(c_i).$$

In the running example, we obtain:

$$AMO(c_1, c_2, c_3, c_4).$$

Compared with the *pairwise* encoding, the *commander* encoding requires fewer clauses but introduces (an acceptable number of) new variables (see Table 1). The *commander* encoding also allows UP to preserve arc consistency.

2.4 The Product Encoding

Chen [14] proposed an AMO encoding which is called *product* encoding. Instead of encoding the AMO constraint $\leq_1(X_1, \dots, X_n)$ he encoded a constraint consisting of n corresponding points, denoted by $\leq_1\{(u_i, v_j) \mid 1 \leq i \leq p, 1 \leq j \leq q, p * q \geq n\}$. The idea can be explained as follows:

1. Each variable $X_k, 1 \leq k \leq n$, is mapped onto a corresponding point (u_i, v_j) , where $u_i \in U = \{u_1, \dots, u_p\}$ and $v_j \in V = \{v_1, \dots, v_q\}$.
2. Then, the *product* encoding is obtained as:

$$AMO(X) = AMO(U) \wedge AMO(V) \bigwedge_{1 \leq k \leq n, k=(i-1)q+j}^{1 \leq i \leq p, 1 \leq j \leq q} ((\bar{X}_k \vee u_i) \wedge (\bar{X}_k \vee v_j)),$$

where $AMO(U)$ and $AMO(V)$ can be encoded by either another encoding or a recursive application of the *product* encoding.

With regard to our running example, we choose $p = 3$, $q = 3$, and use the pairwise encoding for $AMO(U)$ and $AMO(V)$. The derived clauses are:

$$\begin{aligned}
AMO(U) &= (\bar{u}_1 \vee \bar{u}_2) \wedge (\bar{u}_1 \vee \bar{u}_3) \wedge (\bar{u}_2 \vee \bar{u}_3), \\
AMO(V) &= (\bar{v}_1 \vee \bar{v}_2) \wedge (\bar{v}_1 \vee \bar{v}_3) \wedge (\bar{v}_2 \vee \bar{v}_3), \\
AMO(X) &= AMO(U) \wedge AMO(V) \wedge \\
&\quad (\bar{X}_1 \vee u_1) \wedge (\bar{X}_1 \vee v_1) \wedge (\bar{X}_2 \vee u_2) \wedge (\bar{X}_2 \vee v_1) \wedge \\
&\quad (\bar{X}_3 \vee u_3) \wedge (\bar{X}_3 \vee v_1) \wedge (\bar{X}_4 \vee u_1) \wedge (\bar{X}_4 \vee v_2) \wedge \\
&\quad (\bar{X}_5 \vee u_2) \wedge (\bar{X}_5 \vee v_2) \wedge (\bar{X}_6 \vee u_3) \wedge (\bar{X}_6 \vee v_2) \wedge \\
&\quad (\bar{X}_7 \vee u_1) \wedge (\bar{X}_7 \vee v_3) \wedge (\bar{X}_8 \vee u_2) \wedge (\bar{X}_8 \vee v_3).
\end{aligned}$$

One should observe that UP on the *product* encoding achieves arc consistency.

2.5 The Sequential Encoding

By building a count-and-compare hardware circuit and translating this circuit to an equivalent CNF formula, Sinz [38] introduced an encoding of $\leq_k(X_1, \dots, X_n)$ which he called the *sequential* encoding. Here, we only consider the case $k = 1$ and obtain the following clauses:

$$(\bar{X}_1 \vee s_1) \wedge (\bar{X}_n \vee \bar{s}_{n-1}) \bigwedge_{1 < i < n} ((\bar{X}_i \vee s_i) \wedge (\bar{s}_{i-1} \vee s_i) \wedge (\bar{X}_i \vee \bar{s}_{i-1})), \quad (1)$$

where s_i , $1 \leq i \leq n - 1$, are auxiliary variables.

The running example is represented by the following clauses:

$$\begin{aligned}
&\bar{X}_1 \vee s_1, \\
&\bar{X}_2 \vee s_2, \quad \bar{s}_1 \vee s_2, \quad \bar{X}_2 \vee \bar{s}_1, \\
&\bar{X}_3 \vee s_3, \quad \bar{s}_2 \vee s_3, \quad \bar{X}_3 \vee \bar{s}_2, \\
&\bar{X}_4 \vee s_4, \quad \bar{s}_3 \vee s_4, \quad \bar{X}_4 \vee \bar{s}_3, \\
&\bar{X}_5 \vee s_5, \quad \bar{s}_4 \vee s_5, \quad \bar{X}_5 \vee \bar{s}_4, \\
&\bar{X}_6 \vee s_6, \quad \bar{s}_5 \vee s_6, \quad \bar{X}_6 \vee \bar{s}_5, \\
&\bar{X}_7 \vee s_7, \quad \bar{s}_6 \vee s_7, \quad \bar{X}_7 \vee \bar{s}_6, \\
&\bar{X}_8 \vee \bar{s}_7.
\end{aligned}$$

As Marques-Silva and Lynce [37] pointed out, the sequence $s_1, \dots, s_{n-1}$ is of the form "0...01...1" and whenever any Boolean variable X_i is assigned to *TRUE* (or *1*), $1 \leq i \leq n$, consequently, under UP all the other variables X_j must be forced to *FALSE* (or *0*), $1 \leq j \neq i \leq n$. In other words, this encoding allows UP to guarantee the arc consistency property.

2.6 The Ladder Encoding

Gent and Nightingale [22] used the ladder structure, originally proposed by Gent et al. [23], to describe a new encoding of the all-different constraint into SAT. It was named the *ladder* encoding.

Without loosing the correctness property, we consider the sequence $s_1, \dots, s_{n-1}$ to play the role of the sequence $y_1, \dots, y_{n-1}$ in [22] with the condition reversed. Now the ladder validity clauses are represented as follows:

$$\bigwedge_{i=1}^n (\bar{s}_{i-1} \vee s_i), \quad (2)$$

where we set

$$s_0 = 0 \wedge s_n = 1. \quad (3)$$

We add the channelling clauses [22]

$$\bigwedge_{i=1}^n ((s_i \wedge \bar{s}_{i-1}) \leftrightarrow X_i), \quad (4)$$

and adding the conjunction of (2) and (4) we obtain the *ladder* encoding [22]:

$$\bigwedge_{i=1}^n ((\bar{s}_{i-1} \vee s_i) \wedge (\bar{s}_i \vee s_{i-1} \vee X_i) \wedge (\bar{X}_i \vee s_i) \wedge (\bar{X}_i \vee \bar{s}_{i-1})). \quad (5)$$

It is easy to check that the clauses $(\bar{s}_i \vee s_{i-1} \vee X_i)$ occurring in (5) are redundant because they do not affect the correctness of the AMO constraint. After removing these redundant clauses the *ladder* encoding is simplified to:

$$\bigwedge_{i=1}^n ((\bar{s}_{i-1} \vee s_i) \wedge (\bar{X}_i \vee s_i) \wedge (\bar{X}_i \vee \bar{s}_{i-1})). \quad (6)$$

Replacing (6) by (3) yields the *sequential* encoding:

$$(\bar{X}_1 \vee s_1) \wedge (\bar{X}_n \vee \bar{s}_{n-1}) \bigwedge_{1 < i < n} ((\bar{X}_i \vee s_i) \wedge (\bar{s}_{i-1} \vee s_i) \wedge (\bar{X}_i \vee \bar{s}_{i-1})). \quad (7)$$

Furthermore, Prestwich supposed the *relaxed ladder* in [33]. Particularly, instead of adding the condition (4), he added the following condition:

$$\bigwedge_{i=1}^n ((s_i \wedge \bar{s}_{i-1}) \leftarrow X_i). \quad (8)$$

Hence, the *relaxed ladder* encoding is exactly the *ladder* encoding without the redundant clauses. Consequently, the *relaxed ladder* encoding and the *sequential* encoding are identical. Argelich et al. [7] also noticed that the *sequential* encoding is a reformulation of a *regular* encoding [5]. In fact, it is a simple matter to prove that the *regular* encoding and the *ladder* encodings are identical.

Summarizing, we showed that the *relaxed ladder* encoding and the *sequential* encoding are identical. These encodings are obtained from the *ladder* or the *regular* encoding by removing redundant clauses. One should observe that Tamura et al. used the *ladder* structure in the *order* encoding to translate CSPs to SAT instances in their SAT-based solving system [39]. Bailleux et al. [9] also used this structure, called *unary representation*, during their translation of cardinality constraints and pseudo-Boolean constraints to SAT formulas [9,16,10].

Recently, Martins et. al [28] compared both encodings, the *sequential* encoding and the *ladder* encoding. As the result, the experiment shown in their paper indicated very small difference between the two encodings.

In conclusion, here we claim the similarity of the *ladder*, *sequential*, *relaxed ladder*, *regular*, *order* encodings, and the *unary representation*. We hope that this work could help the SAT community to recognize the similarities of these encodings.

3 The Bimander Encoding

The general idea of the new encoding is based on both the ideas of the *binary* encoding and the *commander* encoding. We refer to it as the *bimander* encoding.

Similarly to the *commander* encoding, with a given positive number m , $1 \leq m \leq n$, we partition a set of propositional variables $X = \{X_1, \dots, X_n\}$ into m disjoint subsets $\{G_1, \dots, G_m\}$ such that each subset G_i consists of $g = \lceil \frac{n}{m} \rceil$ variables. However, instead of introducing commander variables like in the *commander* encoding, we introduce a set of auxiliary propositional variables $B_1, \dots, B_{\lceil \log_2 m \rceil}$ like in the *binary* encoding. The variables $B_1, \dots, B_{\lceil \log_2 m \rceil}$ play the role of the commander variables in the *commander* encoding.

The *bimander* encoding is the conjunction of the following clauses:

1. At most *one* variable in each subset can be *TRUE*. We encode this constraint for each subset G_i , $1 \leq i \leq m$, by using the *pairwise* encoding:

$$\bigwedge_{i=1}^m AMO(G_i). \quad (9)$$

In our running example we choose $m = \lceil \sqrt{n} \rceil = 3$ to obtain:

$$AMO(X_1, X_2, X_3) \wedge AMO(X_4, X_5, X_6) \wedge AMO(X_7, X_8).$$

2. The following clauses are generated by the constraints between each variable and commander variables in a subset:

$$\bigwedge_{i=1}^m \bigwedge_{h=1}^g \bigwedge_{j=1}^{\lceil \log_2 m \rceil} (\overline{X}_{i,h} \vee \phi(i, j)), \quad (10)$$

where $\phi(i, j)$ denotes B_j (or $\overline{B_j}$) if bit j of $i - 1$ represented by a unique binary string is 1 (or 0).

The following set of clauses is generated for the running example:

$$\begin{array}{lll}
\overline{X}_1 \vee \overline{B}_1, & \overline{X}_4 \vee B_1, & \overline{X}_7 \vee \overline{B}_1, \\
\overline{X}_1 \vee \overline{B}_2, & \overline{X}_4 \vee \overline{B}_2, & \overline{X}_7 \vee B_2, \\
\overline{X}_2 \vee \overline{B}_1, & \overline{X}_5 \vee B_1, & \overline{X}_8 \vee \overline{B}_1, \\
\overline{X}_2 \vee \overline{B}_2, & \overline{X}_5 \vee \overline{B}_2, & \overline{X}_8 \vee B_2, \\
\overline{X}_3 \vee \overline{B}_1, & \overline{X}_6 \vee B_1, & \\
\overline{X}_3 \vee \overline{B}_2, & \overline{X}_6 \vee \overline{B}_2, &
\end{array}$$

Compared with the *commander* encoding, the *bimander* encoding does not require any constraints among the sequences of auxiliary variables because any combination of such variables $B_1, \dots, B_{\lceil \log_2 m \rceil}$ of a corresponding subset is different from any combinations of all the other groups. Let us prove some important properties of the *bimander* encoding, following the definitions mentioned at the beginning of Section 2.

Correctness Let $\hat{x} = (X_1, \dots, X_l), 1 \leq l \leq n$, be a partial assignment with *at most one* variable assigned to *TRUE*. In case all variables are mapped to *FALSE*, then condition (9) is trivially satisfied. The same holds for condition (10). In case that only one variable, say $X_i, 1 \leq i \leq n$, is mapped to *TRUE*, then there is a corresponding sequence of *TRUE* values assigned to the corresponding sequence of $\{B_1, \dots, B_{\lceil \log_2 m \rceil}\}$. Hence, condition (10) is satisfied as well. Therefore, the partial assignment $\hat{x}$ can possibly be extended to a complete assignment that satisfies the two conditions.

Now suppose that we have a partial assignment $\hat{x} = (X_1, \dots, X_l), 1 \leq l \leq n$, with more than one variable assigned to *TRUE*. Suppose that $X_i = \text{TRUE}$ and $X_j = \text{TRUE}$, for $1 \leq i \neq j \leq l$. In order to satisfy the condition (9), the variables X_i and X_j must belong to different subsets. That leads to two differently corresponding patterns of the sequence $\{B_1, \dots, B_{\lceil \log_2 m \rceil}\}$ which are assigned to *TRUE*. As a result, the sequence contains one propositional variable $B_k, 1 \leq k \leq \lceil \log_2 m \rceil$, that is assigned to both *TRUE* and *FALSE* at the same time, which is impossible. Hence, if any partial assignment has more than one variable assigned to *TRUE*, then UP produces an empty clause. It means that this partial assignment can not be extended to a complete assignment.

In conclusion, the *bimander* encoding correctly encodes AMO.

Propagation Strength Let $\hat{x} = (X_1, \dots, X_l), 1 \leq l \leq n$, be a partial assignment where *TRUE* is assigned to exactly one variable. Now we will show that UP will assign all other variables to *FALSE*. Suppose that variable $X_{i,j} = \text{TRUE}$, which is the j^{th} variable in the subset $G_i, 1 \leq i \leq m$, then this assignment forces a corresponding pattern of the sequence $\{B_1, \dots, B_{\lceil \log_2 m \rceil}\}$ to *TRUE*. Because $X_{i,j} = \text{TRUE}$, all other variables in the subset G_i are set to *FALSE* due to condition (9). Due to condition (10), all the other variables in the subsets $G_{i'}, 1 \leq i' \neq i \leq m$, are set to *FALSE* because they have different patterns of the sequence $\{B_1, \dots, B_{\lceil \log_2 m \rceil}\}$ corresponding to $X_{i,j} = \text{TRUE}$. In conclusion, UP on the *bimander* encoding maintains arc consistency.

Complexity As we mentioned, we need a set of $\lceil \log_2 m \rceil$ auxiliary variables. Condition (9) using the *pairwise* encoding requires $m * \lceil \frac{g(g-1)}{2} \rceil = \frac{n(\frac{n}{m}-1)}{2}$ new clauses. Condition (10) requires $m * \lceil g * \log_2 m \rceil = n * \lceil \log_2 m \rceil$ clauses. Hence, the encoding uses $\frac{n(\frac{n}{m}-1)}{2} + n \lceil \log_2 m \rceil = \frac{n^2}{2m} + n \lceil \log_2 m \rceil - \frac{n}{2}$ clauses.

Generalization It is worth pointing out that the *bimander* encoding can be easily generalized to encode the at-most-k constraint. Again, the set of variables is partitioned into several subsets.

1. For each subset, the at-most-k constraint is encoded by a modified pairwise (or another) encoding.
2. The constraints between each variable and the commander variables in a subset are encoded by the following clauses:

$$\bigwedge_{i=1}^m \bigwedge_{h=1}^g \bigvee_{l=1}^k \bigwedge_{j=1}^{\lceil \log_2 m \rceil} (\overline{X}_{i,h} \vee \phi(i, h, l, j)),$$

where $\phi(i, h, l, j)$ denotes $B_{l,j}$ (or $\overline{B}_{l,j}$) if bit j of $i - 1$ represented by a binary string is 1 (or 0).

Special Cases One should observe that the *bimander* encoding is a general case of several encodings. For example,

- the *pairwise* encoding is a special case of the *bimander* encoding by setting $m = 1$;
- the *commander* encoding is a special case of the *bimander* encoding by setting $m = 2$ (when both encodings divide into 2 subsets); and
- the *binary* encoding is a special case of the *bimander* encoding by setting $m = n$.

4 Comparison and Experimental Evaluation

In this section, we first summarize key features of SAT encodings of the AMO constraint. Thereafter, we experimentally evaluate the encodings presented in Section 3 using different domains.

4.1 Comparison

Table 1 presents the key features of many approaches for encoding the AMO constraint (column *enc*). The columns *clauses* and *aux vars* depict the number of clauses required and auxiliary variables, respectively. The column *AC* indicates whether UP achieves arc consistency. The column *origin* refers to the original publications where the encoding had been introduced. m denotes the disjointed subsets by dividing the set of propositional variables $\{X_1, \dots, X_n\}$ in the *bimander* encoding. In addition to the encodings of the AMO constraint presented in Section 3, we also mention several more encodings that are mainly used for cardinality constraints. As we can see in Table 1, the *bimander* encoding requires the least auxiliary variables among known encodings – with the exception of the *pairwise* encoding. The *totalizer* encoding proposed by Bailleux et al. [9] requires clauses of size at most 3, and the *commander* encoding proposed by Klieber and Kwon [26] needs m (number of disjointed subsets) clauses of size $\lceil \frac{n}{m} + 1 \rceil$, whereas the *product*, *sequential*, *binary* and *bimander* encodings require only binary clauses. This is a considerable advantage of the new encoding.

Table 1. A summary of almost all known encodings of the AMO encoding.

<i>enc</i>	<i>clauses</i>	<i>aux vars</i>	<i>AC</i>	<i>origin</i>
pairwise	$\binom{n}{2}$	0	yes	none
linear	$8n$	$2n$	no	[42]
totalizer	$O(n^2)$	$O(n \log(n))$	yes	[9]
binary	$n \log_2 n$	$\lceil \log_2 n \rceil$	yes	[19]
sequential	$3n - 4$	$n - 1$	yes	[38]
sorting networks	$O(n \log_2^2 n)$	$O(n \log_2^2 n)$	yes	[16]
commander	$\sim 3n$	$\sim \frac{n}{2}$	yes	[26]
product	$2n + 4\sqrt{n} + O(\sqrt[4]{n})$	$2\sqrt{n} + O(\sqrt[4]{n})$	yes	[14]
card. networks	$6n - 9$	$4n - 6$	yes	[8]
PHFs-based	$n \log_2 n$	$\lceil \log_2 n \rceil$	yes	[11]
bimander	$\frac{n^2}{2m} + n \log_2 m - \frac{n}{2}$	$\log_2 m, 1 \leq m \leq n$	yes	this paper
bimander ($m = \frac{n}{2}$)	$n \log_2 n - \frac{n}{2}$	$\lceil \log_2 n \rceil - 1$	yes	this paper

4.2 Experimental Evaluation

For the experimental evaluation we have selected some well-known problems which have been used in recent CSP and SAT competitions. In case of the *bimander* encoding, we have considered two different values for the parameter m , viz. $m = \sqrt{n}$ and $m = \frac{n}{2}$.

Our experiments were conducted using CLASP 2 [20] with default configuration on a 2.66-GHz Intel Core 2 Quad processor with 3.8 GB of memory. Bold font indicates the minimum time for each benchmark. We abbreviate *pairwise*, *sequential*, *commander*, *binary*, *product*, and *bimander* encodings as *pw*, *seq*, *cmd*, *bin*, *pro* and *bim*, respectively. For the *commander* encoding, the set of variables is recursively divided into 2 disjoint subsets since the encoding in that case conducted on our problems gives a best result in term of the average time.

Pigeon-Hole Problems The goal of the problem is to prove that p pigeons can not fit in $h = p - 1$ holes. Table 2 shows the results from different encodings on unsatisfiable Pigeon-Holes instances.

Table 2. A comparison of the run times for Pigeon-Hole problems. Run times are in seconds.

<i>enc</i>	<i>pw</i>	<i>seq</i>	<i>cmd</i>	<i>bin</i>	<i>pro</i>	<i>bim</i> ($\sqrt{n}$)	<i>bim</i> ($n/2$)
10	2.16	0.73	0.56	0.80	0.22	0.33	0.22
11	22.15	5.79	4.46	6.59	6.13	5.10	2.10
12	244.59	117.83	43.27	29.52	43.21	38.19	26.06
13	> 3600.00	1604.14	352.53	142.60	736.25	546.91	64.91
14	> 3600.00	> 3600.00	> 3600.00	1271.24	> 3600.00	> 3600.00	560.03
<i>average</i>	> 1493.78	> 1065.69	> 800.16	290.15	> 877.16	> 838.10	130.66

It can be seen that the *bimander* encoding (with $m = \frac{n}{2}$) performs best in all cases, followed by the *binary* encoding, whereas the *bimander* encoding (with $m = \sqrt{n}$) outperforms the rest.

Hamiltonian Cycle Problems Given a directed or undirected graph with a set of vertices and a set of edges between vertices. A Hamiltonian cycle is a path that visits each vertex exactly once, except for the vertex that is both first and the last, which is visited twice. The different instances were taken from [1].

The run times for the different encodings of the Hamiltonian cycle problem are presented in Table 3. Both *bimander* encodings achieve the best average run time, followed by the *binary* encoding. The *commander* and *product* encodings are faster than the *sequential* encoding, whereas *pairwise* encoding performs worse.

Table 3. A comparison of run times for satisfiable Hamiltonian cycle instances. Run times are in seconds.

<i>enc</i>	<i>pw</i>	<i>seq</i>	<i>cmd</i>	<i>bin</i>	<i>pro</i>	<i>bim</i> ($\sqrt{n}$)	<i>bim</i> ($n/2$)
miles750	135.48	25.42	13.67	38.19	14.18	32.55	22.92
miles1000	67.77	10.93	7.65	7.38	12.45	9.52	8.19
miles1500	30.01	3.30	2.60	2.95	2.46	3.74	3.16
queen10_10	13.87	4.16	3.54	3.77	3.68	4.00	3.75
queen11_11	32.34	9.75	8.32	8.43	8.41	9.23	8.16
queen12_12	1.73	22.46	20.13	21.49	18.43	20.44	21.13
queen13_13	3.10	40.99	38.43	1.58	36.30	1.45	1.39
queen14_14	5.17	2.47	2.53	2.42	2.09	2.27	2.20
queen15_15	7.75	3.64	3.42	3.76	3.17	3.47	3.37
queen16_16	11.26	4.80	5.21	5.44	5.14	5.37	5.25
<i>average</i>	30.84	12.79	10.55	9.54	10.63	9.20	7.95

All-Interval Series Problems The goal of the problem is to arrange a permutation of the n integers ranging from 1 to n in such a way that the differences between adjacent numbers are also a permutation, of the numbers from 1 to $n - 1$. As a result, the performance of this benchmarks is heavily influenced by the encoding of the AMO constraint. All-interval series problems are one of the classical CSPs and are usually regarded as a difficult benchmark to find all solutions (see prob007 in [24]).

Table 4. A comparison of run times for All-Interval Series problems. Run times are in seconds. *sol* shows the number of all solutions of the corresponding instance.

<i>enc</i>	<i>pw</i>	<i>seq</i>	<i>cmd</i>	<i>bin</i>	<i>pro</i>	<i>bim</i> ($\sqrt{n}$)	<i>bim</i> ($n/2$)	<i>sol</i>
7	0.05	0.03	0.02	0.02	0.05	0.01	0.02	32
8	0.56	1.07	0.63	0.20	0.49	0.62	0.62	40
9	5.33	8.92	0.37	0.27	5.61	0.33	0.24	120
10	61.72	104.02	1.72	1.58	60.71	1.95	1.46	296
11	972.54	1387.67	11.96	8.94	269.43	11.34	6.72	648
12	> 3600.00	> 3600.00	78.91	49.24	> 3600.00	69.52	43.81	1328
13	> 3600.00	> 3600.00	517.72	356.64	> 3600.00	504.61	276.34	3200
14	> 3600.00	> 3600.00	3200.21	2748.69	> 3600.00	3537.74	2005.18	9912
<i>average</i>	> 1480.02	> 1537.71	476.44	395.69	> 1392.03	515.76	291.79	

Table 4 summaries the run times for different encodings on AIS instances. Except for the cases $n = 7$ and $n = 8$, the table shows that the *bimander* encoding with $m = \frac{n}{2}$ significantly surpasses all the others. Moreover, for the final three instances this one performs in a reasonable time, whereas the *pairwise*, *sequential*, and *product* encodings carry out more than 3600 seconds. The *binary* encoding gives rather good results, while the *bimander* encoding in case $m = \sqrt{n}$ and the *commander* encoding perform similarly. The *pairwise*, *sequential*, and *product* encodings perform worse.

Quasigroup With Holes Problems A quasigroup is a square of values x_{ij} , $1 \leq i, j \leq n$, where each number $[1..n]$ occurs exactly once in each row and column. Achlioptas et al. [2] introduced an encoding for generating satisfiable quasigroups with holes instances in which some cells are filled. Quasigroup with holes instances can be consider as a multiple permutation problem in which the variables may occur in more than one permutation problem. Moreover, the encoding can tune the generator to output hard instances. We experimented with instances with different levels of hardness.

Table 5 shows the results from different encodings on satisfiable quasigroup with holes problems. The *bimander* encoding with parameter $m = \sqrt{n}$ is clearly the fastest with the exception of the instance qwh.order40.holes544. Surprisingly, the *pairwise* encoding performs very well followed by the *commander* encoding. The *bimander* encoding with parameter $m = \frac{n}{2}$, the *binary*, and the *product* encoding are quite similar. Although the *sequential* encoding was the fastest on the instance qwh.order40.holes544, its overall performance is poor.

Table 5. A comparison of run times for satisfiable quasigroup with holes problems. Run times are in seconds.

<i>enc</i>	<i>pw</i>	<i>seq</i>	<i>cmd</i>	<i>bin</i>	<i>pro</i>	<i>bim</i> ($\sqrt{n}$)	<i>bim</i> ($n/2$)
qwh.order30.holes320	0.46	0.28	0.23	0.25	0.23	0.20	0.22
qwh.order35.holes405	3.62	3.51	10.35	6.51	5.73	1.60	2.14
qwh.order40.holes528	134.71	115.62	124.26	120.47	241.20	58.90	159.21
qwh.order40.holes544	39.26	14.57	47.82	123.72	46.7	70.81	154.03
qwh.order40.holes560	121.74	65.36	55.68	119.66	33.16	21.22	53.27
qwh.order33.holes381	58.73	435.90	174.29	94.22	108.03	12.74	92.30
<i>average</i>	358.52	635.24	412.63	464.83	435.05	165.47	461.17

5 Conclusions and Future Works

Inspired by being remarkably successful at solving hard and practical problems of SAT solving, many problems that were solved previously by other encodings can now be solved more effectively by translating them into SAT instances and applying advanced SAT solvers to find solutions. During the encoding phase, one of the most important constraints occurring naturally in a wide range of real world applications, is the AMO constraint. Hence, many problems may benefit from effective encodings of this constraint.

The paper has four contributions. Firstly, we pointed out that the *ladder* encoding [22] is nothing but the *sequential* encoding [38] and a set of redundant clauses.

Moreover, the *relaxed ladder* encoding [33] and the *sequential* encoding are identical. Another two encodings, viz. the *ladder* and the *regular* [5] encoding, are also identical. Moreover, the *relaxed ladder* and the *sequential* encodings can be obtained from the *ladder* and the *regular* encoding after removing redundant clauses. Interestingly, these ideas were exploited in the *unary representation* [9] and the *order* encoding [39].

Secondly, we proposed a new encoding for AMO, the so-called *bimander* encoding. Compared to many other well-known AMO encodings, the *bimander* encoding requires the least auxiliary variables (with the exception of the *pairwise* encoding which does not require any auxiliary variables at all). Although the *commander* encoding and the *bimander* encoding use the same approach by dividing the original set of propositional variables, the *commander* encoding requires clauses of size $\lceil \frac{n}{m} + 1 \rceil$ (where m is the number of disjoint subsets), whereas the *bimander* encoding requires only binary clauses. We believe that this helps the *bimander* encoding to perform better than the *commander* encoding in our experimental evaluation. Moreover, the *bimander* encoding has the advantage of high scalability, and it can easily be adjusted in terms of the number of additional propositional variables to obtain particular encodings. For example, the *pairwise* or *binary* encodings are special cases of the *bimander* encoding.

Thirdly, this paper also proposes a special case, when dividing the propositional variables into $m = \lceil \frac{n}{2} \rceil$ disjoint subsets. From a theoretical point of view, this case is better than the *binary* encoding due to fewer auxiliary variables and clauses. From a practical point of view, we show that this special case of the *bimander* encoding ($m = \lceil \frac{n}{2} \rceil$) performs better than the *binary* encoding in all experiments in term of run time.

Fourthly, in practice, the *bimander* encoding is practical and easy to implement. Our results reveal that two particular cases of the *bimander* encoding are very competitive in a comparison with other well-known encodings.

A future research is to study how the number of disjoint subsets could affect the *bimander* encoding in realistic problems. It would be particularly useful to extend our findings to the at-most-k constraint. Finally, the ultimate goal should carry out a profound study of not only analytical, but also theoretical knowledge of variants of well-known encodings. We expect that this will help us to deepen our understanding on what encoding one should select given a particular problem.

Acknowledgments We would like to thank Christoph Wernhard, Norbert Manthey for many fruitful suggestions, and Martin Gebser for his helpful discussions. We also wish to thank Carla Gomes for her kindly providing us the QWH's generator.

References

1. A Computational Symposium at Cornell University, Ithaca, NY, USA, 2002. <http://mat.gsia.cmu.edu/COLOR03/>
2. Achlioptas, D., Gomes, C.P., Kautz, H.A., Selman, B.: Generating Satisfiable Problem Instances. In: Kautz, H.A., Porter, B.W. (eds.) Proceedings of the Seventeenth National Conference on Artificial Intelligence and Twelfth Conference on Innovative Applications of Artificial Intelligence, July 30 - August 3, 2000, Austin, Texas, USA. pp. 256–261. AAAI Press / The MIT Press (2000)

3. Anbulagan, Grastien, A.: Importance of variables semantic in CNF encoding of cardinality constraints. In: Bulitko, V., Beck, J.C. (eds.) Eighth Symposium on Abstraction, Reformulation, and Approximation, SARA 2009, Lake Arrowhead, California, USA, 8-10 August 2009. AAAI (2009)
4. Ansótegui, C., Bonet, M.L., Levy, J.: Solving (weighted) partial maxsat through satisfiability testing. In: Kullmann, O. (ed.) Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings. pp. 427–440. Lecture Notes in Computer Science, Springer (2009)
5. Ansótegui, C., Manyà, F.: Mapping problems with finite-domain variables into problems with boolean variables. In: SAT 2004 - The Seventh International Conference on Theory and Applications of Satisfiability Testing, 10-13 May 2004, Vancouver, BC, Canada, Online Proceedings. pp. 1–15. Springer LNCS (2004)
6. Argelich, J., Cabiscol, A., Lynce, I., Manyà, F.: Sequential Encodings from Max-CSP into Partial Max-SAT. In: Kullmann, O. (ed.) Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings. Lecture Notes in Computer Science, vol. 5584, pp. 161–166. Springer (2009)
7. Argelich, J., Cabiscol, A., Lynce, I., Manyà, F.: New Insights into Encodings from MaxCSP into Partial MaxSAT. In: 40th IEEE International Symposium on Multiple-Valued Logic, ISMVL 2010, Barcelona, Spain, 26-28 May 2010. pp. 46–52. IEEE Computer Society (2010)
8. Asín, R., Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E.: Cardinality Networks: a Theoretical and Empirical Study. Constraints 16(2), 195–221 (2011)
9. Bailleux, O., Bouffkhad, Y.: Efficient CNF Encoding of Boolean Cardinality Constraints. Principles and Practice of Constraint Programming 9th International Conference CP-2003 pp. 108–122 (2003)
10. Bailleux, O., Bouffkhad, Y., Roussel, O.: New Encodings of Pseudo-Boolean Constraints into CNF. In: Kullmann, O. (ed.) Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings. Lecture Notes in Computer Science, vol. 5584, pp. 181–194. Springer (2009)
11. Ben-Haim, Y., Ivrii, A., Margalit, O., Matsliah, A.: Perfect Hashing and CNF Encodings of Cardinality Constraints. In: Cimatti, A., Sebastiani, R. (eds.) Theory and Applications of Satisfiability Testing - SAT 2012 - 15th International Conference, Trento, Italy, June 17-20, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7317, pp. 397–409. Springer (2012)
12. Bessiere, C.: Chapter 3 Constraint Propagation, vol. 2, pp. 27 – 81. Elsevier (2006)
13. Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.): Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, vol. 185. IOS Press (2009)
14. Chen, J.C.: A new SAT Encoding of the At-Most-One Constraint. In: Proc. of the Tenth Int. Workshop of Constraint Modelling and Reformulation (2010)
15. Davis, M., Logemann, G., Loveland, D.: A Machine Program for Theorem-proving. Commun. ACM 5(7), 394–397 (Jul 1962)
16. Eén, N., Sörensson, N.: Translating Pseudo-Boolean Constraints into SAT. Journal on Satisfiability, Boolean Modeling and Computation 2, 1–26 (2006)
17. Frisch, A.M., Giannoros, P.A.: SAT Encodings of the At-Most-k Constraint. Some Old, Some New, Some Fast, Some Slow. In: Proc. of the Tenth Int. Workshop of Constraint Modelling and Reformulation (2010)
18. Frisch, A.M., Peugniez, T.J., Doggett, A.J., Nightingale, P.W.: Solving Non-Boolean Satisfiability Problems with Stochastic Local Search. In: in Proc. IJCAI-01. pp. 282–288 (2001)
19. Frisch, A.M., Peugniez, T.J., Doggett, A.J., Nightingale, P.W.: Solving Non-Boolean Satisfiability Problems with Stochastic Local Search: A Comparison of Encodings. J. Autom. Reason. 35, 143–179 (October 2005)

20. Gebser, M., Kaufmann, B., Schaub, T.: The Conflict-Driven Answer Set Solver clasp: Progress Report. In: Erdem, E., Lin, F., Schaub, T. (eds.) *Logic Programming and Non-monotonic Reasoning*, 10th International Conference, LPNMR 2009, Potsdam, Germany, September 14-18, 2009. *Proceedings. Lecture Notes in Computer Science*, vol. 5753, pp. 509–514. Springer (2009)
21. Gent, I.: Arc Consistency in SAT. In *fifteenth European Conference on Artificial Intelligence* pp. 121–125 (2002)
22. Gent, I., Nightingale, P.: A New Encoding of AllDifferent into SAT. In: Frisch, A.M., Miguel, I. (eds.) *Proceedings 3rd International Workshop on Modelling and Reformulating Constraint Satisfaction Problems*. pp. 95–110. Springer (2004)
23. Gent, I., Prosser, P., Smith, B.: A 0/1 encoding of the gaclex constraint for pairs of vectors. In: *ECAI 2002 workshop W9: Modelling and Solving Problems with Constraints*. University of Glasgow (2002)
24. Hnich, B., Miguel, I., Gent, I.P., , Walsh, T.: CSPLib is a Library of Test Problems for Constraint Solvers. <http://www.csplib.org/>, [Online; accessed 24-April-2013]
25. Kautz, H., Selman, B.: The State of SAT. *Discrete Appl. Math.* 155, 1514–1524 (June 2007)
26. Klieber, W., Kwon, G.: Efficient CNF Encoding for Selecting 1 from N Objects . In: the *Fourth Workshop on Constraint in Formal Verification (CFV)* (2007)
27. Marques-silva, J.: Practical Applications of Boolean Satisfiability. In: *In Workshop on Discrete Event Systems (WODES)*. IEEE Press (2008)
28. Martins, R., Manquinho, V.M., Lynce, I.: Exploiting Cardinality Encodings in Parallel Maximum Satisfiability. In: *IEEE 23rd International Conference on Tools with Artificial Intelligence, ICTAI 2011, Boca Raton, FL, USA, November 7-9, 2011*. pp. 313–320 (2011)
29. Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S.: Chaff: Engineering an Efficient SAT Solver. In: *Proceedings of the 38th Design Automation Conference, DAC 2001, Las Vegas, NV, USA, June 18-22, 2001*. pp. 530–535. ACM (2001)
30. Prestwich, S.D.: Local Search on SAT-Encoded Colouring Problems. In: *Lecture Notes in Computer Science*. vol. 2919. Springer Verlag (2003)
31. Prestwich, S.D.: SAT Problems with Chains of Dependent Variables. *Discrete Applied Mathematics* 130(2), 329–350 (2003)
32. Prestwich, S.D.: Full Dynamic Substitutability by SAT Encoding. In: Wallace, M. (ed.) *Principles and Practice of Constraint Programming - CP 2004, 10th International Conference, September 27 - October 1, 2004, Toronto, Canada. Lecture Notes in Computer Science*, vol. 3258, pp. 512–526. Springer (2004)
33. Prestwich, S.D.: Finding Large Cliques using SAT Local Search. vol. *Trends in Constraint Programming*, pp. 273–278. ISTE (2007)
34. Prestwich, S.D.: Variable Dependency in Local Search: Prevention Is Better Than Cure. In: Silva, J.M., Sakallah, K.A. (eds.) *Theory and Applications of Satisfiability Testing - SAT 2007, 10th International Conference, Lisbon, Portugal, May 28-31, 2007, Proceedings. Lecture Notes in Computer Science*, vol. 4501, pp. 107–120. Springer (2007)
35. Prestwich, S.D.: CNF Encodings. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) *Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications*, vol. 185, pp. 75–97. IOS Press (2009)
36. Rossi, F., Beek, P.v., Walsh, T.: *Handbook of Constraint Programming (Foundations of Artificial Intelligence)*. Elsevier Science Inc., New York, NY, USA (2006)
37. Silva, J.M., Lynce, I.: Towards Robust CNF Encodings of Cardinality Constraints . In: *Proc. 13th International Conference on Principles and Practice of Constraint Programming CP-2007. Lecture Notes in Computer Science*, vol. 4741, pp. 483–497. Springer (2007)
38. Sinz, C.: Towards an Optimal CNF Encoding of Boolean Cardinality Constraints. In: *Principles and Practice of Constraint Programming, 11th International Conference, CP 2005,*

- Spain, October 2005, Proceedings. Lecture Notes in Computer Science, vol. 3709, pp. 827–831. Springer (2005)
39. Tamura, N., Taga, A., Kitagawa, S., Banbara, M.: Compiling Finite Linear CSP into SAT. *Constraints* 14(2), 254–272 (2009)
 40. Velev, M.N.: Exploiting Hierarchy and Structure to Efficiently Solve Graph Coloring as SAT. In: International Conference on Computer-Aided Design (ICCAD’07), November 5–8, 2007, San Jose, CA, USA. pp. 135–142. IEEE (2007)
 41. Walsh, T.: SAT v CSP. In: Principles and Practice of Constraint Programming - CP2000. Lecture Notes in Computer Science, vol. 1894, pp. 441–456. Springer (2000)
 42. Warners, J.P.: A Linear-Time Transformation of Linear Inequalities into Conjunctive Normal Form. *Information Processing Letters* 68(2), 63–69 (1998)